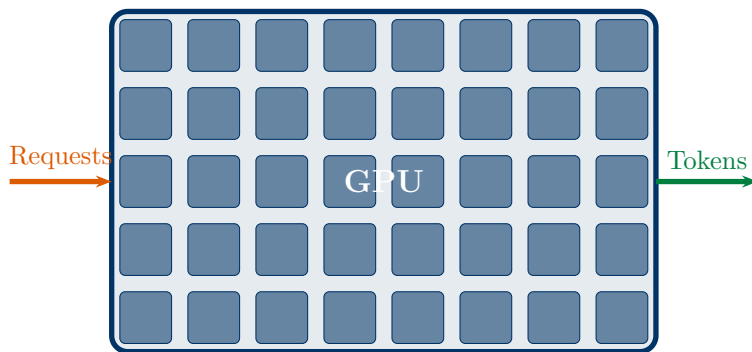


AI Inference Engineering

Building Fast, Scalable, and
Production-Ready AI Systems

AI Inference Engineering

Building Fast, Scalable, and Production-Ready AI Systems



From First Principles to Production Systems

Covers: GPU Architecture · CUDA · Transformers · KV Cache · Quantization
vLLM · TensorRT-LLM · SGLang · Speculative Decoding · Distributed Inference
Kubernetes · Observability · NVIDIA H100/Blackwell · 2025–2026

AI Inference Engineering: Building Fast, Scalable, and Production-Ready AI Systems

Edition 1.0 — May 2026

This book covers GPU inference engineering from first principles through production deployment. It incorporates the latest developments in NVIDIA Blackwell architecture (B200/GB200), vLLM PagedAttention, TensorRT-LLM v1.2, SGLang, speculative decoding, FP8/FP4 quantization, and NVIDIA Dynamo disaggregated serving.

For educational purposes.

Typeset with L^AT_EX / TeX Live 2023.

Contents

Preface	9
I Foundations	11
1 Why Inference Engineering Matters	13
1.1 The Two Lives of an AI Model	13
1.2 The Inference Problem	14
1.3 What Inference Engineers Do	14
1.4 The Three Metrics That Matter	15
1.5 A Preview of the Journey	16
2 GPU Architecture and CUDA Fundamentals	19
2.1 Why GPUs? The Parallel Computation Story	19
2.2 GPU Hardware Architecture	20
2.2.1 Streaming Multiprocessors (SMs)	20
2.2.2 The GPU Memory Hierarchy	21
2.2.3 Tensor Cores: The Matrix Multiplication Engines	22
2.3 CUDA Programming Model	23
2.3.1 The Thread Hierarchy	23
2.3.2 A Simple CUDA Kernel	24
2.3.3 Memory Management in CUDA	25
2.3.4 Warps: The Real Execution Unit	26

2.4	FlashAttention: The Kernel That Changed Inference	26
3	Transformer Inference Mechanics	29
3.1	How Transformers Generate Text	29
3.1.1	The Two Phases in Detail	30
3.2	The Roofline Model: Compute vs. Memory Bound	30
3.3	Inside a Transformer Layer	31
3.3.1	The Attention Computation	32
3.3.2	Memory Footprint of a Large Model	32
4	KV Cache and Memory Management	35
4.1	Why We Need a KV Cache	35
4.2	KV Cache Memory Calculation	36
4.3	The Problem: KV Cache Fragmentation	36
4.4	PagedAttention: Solving Fragmentation	37
4.5	Prefix Caching	38
4.6	KV Cache Quantization	39
5	Quantization: Fitting More Into Less Memory	41
5.1	What Is Quantization?	41
5.2	Number Formats: A Visual Guide	42
5.3	Post-Training Quantization Methods	43
5.3.1	GPTQ: Round-to-Nearest with Correction	43
5.3.2	AWQ: Activation-Aware Weight Quantization	44
5.3.3	FP8 Quantization on H100/Blackwell	45
5.4	Quantization Accuracy Tradeoffs	45
6	Speculative Decoding	47

6.1	The Bottleneck: Sequential Decode Steps	47
6.2	How Speculative Decoding Works	48
6.2.1	N-gram and EAGLE Speculation	48
7	Inference Runtimes: vLLM, TensorRT-LLM, and SGLang	51
7.1	The Inference Runtime Stack	51
7.2	vLLM: The Production Standard	52
7.2.1	Continuous Batching	52
7.2.2	Running vLLM	53
7.3	TensorRT-LLM: Maximum NVIDIA Performance	54
7.4	SGLang: Structured Generation Language	55
7.5	Runtime Comparison: The Decision Guide	56
8	Distributed Inference: Parallelism Strategies	59
8.1	When One GPU Is Not Enough	59
8.2	Tensor Parallelism	59
8.2.1	Launching Tensor-Parallel vLLM	60
8.3	Pipeline Parallelism	61
8.4	NVLink: The Fast Lane Between GPUs	61
8.5	Disaggregated Prefill and Decode	62
9	Observability: Monitoring Inference Systems	63
9.1	The Three Pillars of Observability	63
9.2	Key Metrics to Monitor	63
9.3	GPU Profiling Tools	64
9.3.1	nvidia-top: Real-Time GPU Monitor	64
9.3.2	Nsight Systems: Timeline Profiling	65

9.3.3	Nsight Compute: Kernel-Level Analysis	65
9.4	Prometheus and Grafana for Inference	65
10	Production Deployment with Kubernetes	67
10.1	Containerizing a vLLM Server	67
10.2	Kubernetes Deployment	68
10.3	Horizontal Pod Autoscaler	69
II	Advanced Topics	73
11	Long Context, Multimodal, and Agentic Inference	75
11.1	Long-Context Inference	75
11.2	Multimodal Inference	76
11.3	Agentic Inference	77
A	CUDA Cheat Sheet	79
B	Linux Commands for AI Engineers	83
C	Benchmarking Toolkit	85
D	GPU Profiling Tools	89
E	Research Papers Every Inference Engineer Should Read	93
	Index	97

Preface

"The best way to learn is to teach."

This book exists because AI is no longer just a research topic — it is an engineering discipline. Every time you ask a question to ChatGPT, Claude, or Gemini, thousands of GPU calculations happen in milliseconds. Behind that seamless experience is a carefully engineered system: one that balances speed, cost, memory, and reliability at a massive scale.

AI Inference Engineering teaches you how to build and operate those systems.

Who this book is for

- Software engineers wanting to understand AI infrastructure
- ML engineers who want to go beyond model training
- Platform engineers deploying AI at scale
- Students learning GPU systems and distributed computing
- Anyone curious about what happens after a model is trained

What you will learn

You will journey from the very first question — *what is a GPU?* — all the way to operating a multi-node, multi-GPU inference cluster serving millions of requests per day. Every concept is built from scratch using simple language and real-world analogies before advancing to production-grade engineering detail.

How to read this book

Read it linearly the first time. Each chapter builds on the previous one. Revisit individual chapters as reference when working on production systems. Run the code examples — intuition comes from doing.

A note on versions

The field moves fast. This edition reflects the state of the art as of mid-2026: NVIDIA H100 and Blackwell (B200/GB200) GPUs, vLLM v0.7+, TensorRT-LLM v1.2, SGLang v0.4+, and Kubernetes-based deployment patterns. Core concepts — batching, memory management, quantization, parallelism — are timeless.

PART I

Foundations

Why Inference Engineering Matters

Learning Objectives

- Understand the difference between AI training and AI inference
- Learn why inference efficiency is the key challenge for production AI
- Grasp the scale of modern AI inference systems
- Appreciate the engineering disciplines that make AI products work

1.1 The Two Lives of an AI Model

Every AI model lives two lives.

In its **first life**, the model learns. Enormous clusters of GPUs process billions of text samples, adjusting billions of numerical parameters over weeks or months. This is called **training**.

In its **second life**, the model works. It answers questions, writes code, summarizes documents, and assists doctors, lawyers, and students around the world. This is called **inference**.

Training is a one-time event. Inference happens billions of times a day.

Analogy: The Chef and the Restaurant

Imagine a chef who spends years learning to cook — practising recipes, mastering techniques, developing a personal style. That long learning period is *training*.

Now the restaurant opens. Every day the chef must cook hundreds of meals, each one served in minutes. That daily cooking is *inference*. The chef trains once. The restaurant runs forever.

1.2 The Inference Problem

When an AI model answers a question, it performs billions of mathematical operations for every sentence it generates. A single forward pass through a 70-billion-parameter model involves roughly 140 billion floating-point multiplications.

Now multiply that by the users:

- ChatGPT processes over 100 million queries per day.
- A single query might generate 200–1000 tokens.
- Each token requires a full model forward pass.

This means AI companies run **trillions** of mathematical operations every second just to keep their services alive. If those operations are not performed efficiently, one of two things happens: the service becomes unbearably slow, or the cost becomes unbearably high.

Key Insight

NVIDIA’s 2025 Annual Report explicitly noted that “inference workloads surpassed training” in revenue terms. The industry has shifted: getting models to run efficiently is now more important commercially than making them more accurate.

1.3 What Inference Engineers Do

An inference engineer’s job is to make AI fast, cheap, and reliable at scale. This involves:

- **Hardware selection:** Which GPUs to buy or rent, and how many
- **Runtime optimization:** Making each GPU operation as fast as possible
- **Memory management:** Fitting large models into limited GPU memory
- **Batching:** Serving many users simultaneously from one GPU
- **Quantization:** Compressing model weights to use less memory and run faster
- **Deployment:** Packaging and scaling inference services on Kubernetes
- **Observability:** Monitoring latency, throughput, and cost in production

1.4 The Three Metrics That Matter

Every inference system is measured by three core metrics:

Latency How long does the user wait? Specifically, *Time to First Token (TTFT)* — the delay before the first word appears — and *Time per Output Token (TPOT)* — how quickly subsequent words arrive.

Throughput How many requests can the system process per second? Higher throughput means lower cost per query.

Memory How much GPU memory does the system use? Memory limits how large a model you can serve and how many requests you can handle simultaneously.

These three metrics form a **triangle of tradeoffs**. Improving one often hurts another. A large batch size improves throughput but increases latency. Quantization reduces memory but may reduce accuracy. Understanding these tradeoffs is the core skill of an inference engineer.

1.5 A Preview of the Journey

This book walks through the entire inference engineering stack, layer by layer:

Chapter 10: Production Deployment & Kubernetes

Chapter 9: Observability & Profiling

Chapter 8: Distributed Inference

Chapter 7: Inference Runtimes (vLLM, TRT-LLM, SGLang)

Chapter 6: Speculative Decoding

Chapter 5: Quantization

Chapter 4: KV Cache & Memory Management

Chapter 3: Transformer Inference Mechanics

Chapter 2: GPU Architecture & CUDA

Chapter 1: Why Inference Engineering Matters (this chapter)

Each layer depends on the one below it. Build a firm foundation and the rest becomes intuitive.

Chapter Summary

AI inference is the act of running a trained model to answer user queries. It is performed billions of times daily and requires careful engineering to be fast and affordable. Three key metrics — latency, throughput, and memory — govern every decision an inference engineer makes. This book builds your knowledge from the GPU chip upward to a full production deployment.

Exercise 1.1: Reflection

Before reading further, write down three AI products you use daily. For each one, estimate: How many users might use it simultaneously? What latency would feel “good enough” to you? Keep these numbers in mind as you read — they will help you appreciate why every optimization in this book matters.

GPU Architecture and CUDA Fundamentals

Learning Objectives

- Understand the fundamental difference between CPUs and GPUs
- Learn GPU memory hierarchy: registers, shared memory, HBM
- Understand CUDA programming model: threads, blocks, grids
- Grasp how Tensor Cores accelerate matrix operations
- Understand NVIDIA's GPU generations: Volta through Blackwell

2.1 Why GPUs? The Parallel Computation Story

A CPU (Central Processing Unit) is designed for sequential, versatile work. It has a small number of very powerful cores — typically 8 to 64 — each capable of complex decision-making, branch prediction, and running operating systems.

A GPU (Graphics Processing Unit) is built for a completely different purpose: doing the *same simple operation thousands of times simultaneously*.

Analogy: One Expert vs. One Thousand Students

Imagine you need to add the numbers on 10,000 pieces of paper.

A **CPU** is like one expert accountant. Fast, smart, capable of handling complex calculations. But it works through papers one at a time — or maybe a few at

a time with multiple cores.

A **GPU** is like 10,000 students with calculators, each working on one paper simultaneously. Each student is slower and less capable than the expert. But together, they finish the whole pile in the time the expert handles one.

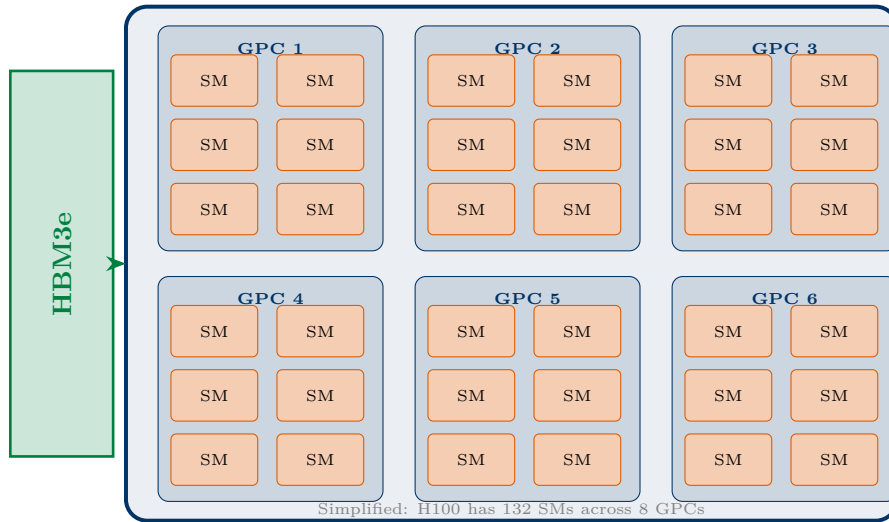
For AI, which involves performing the same multiplication millions of times across different data, the 10,000 students win — by orders of magnitude.

2.2 GPU Hardware Architecture

2.2.1 Streaming Multiprocessors (SMs)

The basic computational unit of a GPU is the **Streaming Multiprocessor** (SM). An NVIDIA H100 GPU has 132 SMs. Each SM contains:

- **CUDA cores:** General-purpose floating-point and integer compute units
- **Tensor Cores:** Specialized matrix-multiply units (described below)
- **Registers:** Tiny, ultra-fast memory local to each thread
- **Shared Memory / L1 Cache:** Fast memory shared across threads in one SM
- **Warp Schedulers:** Hardware that manages thread execution



2.2.2 The GPU Memory Hierarchy

Memory in a GPU is organized in layers, from fastest (but tiny) to slowest (but huge):

Memory Type	Size	Bandwidth	Purpose
Registers	256 KB/SM	~20 TB/s	Per-thread data
L1/Shared Memory	228 KB/SM	~19 TB/s	Thread cooperation within SM
L2 Cache	50 MB	~12 TB/s	Cross-SM caching
HBM3e (VRAM)	80–141 GB	3.35 TB/s	Model weights, KV cache
CPU RAM (via NVLink/PCIe)	TBs	900 GB/s	Offload, overflow

Table 2.1: H100 GPU Memory Hierarchy

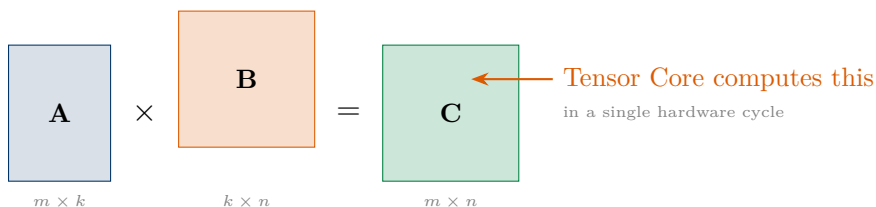
The Golden Rule of GPU Programming

The fastest code is code that keeps data as close to the compute units as possible. If your CUDA kernel reads from HBM when it could read from shared memory, you are leaving 5× performance on the table. Every serious GPU optimization is, at its heart, a memory access optimization.

2.2.3 Tensor Cores: The Matrix Multiplication Engines

Tensor Cores are specialized hardware units inside each SM designed for one specific task: ultra-fast matrix multiplication. They were introduced in the Volta architecture (2017) and have been enhanced in every generation since.

Why does matrix multiplication matter for AI? Because the core operation in every neural network layer is a **matrix multiplication** (GEMM — General Matrix Multiply). When you multiply an input vector by a weight matrix to produce an output, that is a matrix multiply. Modern LLMs perform this operation hundreds of times per forward pass.



The H100 Tensor Core operates on **FP8** precision natively, performing a $16 \times 8 \times 16$ matrix multiplication in a single instruction — the equivalent of 2048 floating-point multiply-add operations per cycle.

GPU Generations and Precision Support

GPU	Arch.	Year	Peak (FP8)	Key Features
A100	Ampere	2020	624 TFLOPS	First FP16 Tensor Cores, NVLink 3
H100 SXM	Hopper	2022	3958 TFLOPS	FP8 native, Transformer Engine, NVLink 4
H200 SXM	Hopper+	2024	3958 TFLOPS	141 GB HBM3e memory
B200	Blackwell	2025	9000 TFLOPS	FP4, 5th-gen Tensor Cores, 30× H100 inference
GB200 NVL72	Blackwell	2025	~1 EFLOPS	72 GPUs in a rack, disaggregated serving

Table 2.2: NVIDIA GPU Generations for AI Inference**Blackwell's Leap**

The NVIDIA B200 (Blackwell) delivers up to 30× the inference performance of the H100 on specific workloads, particularly for large models using FP4 quantization. The fifth-generation Tensor Cores support FP32, FP16, BF16, FP8, INT8, FP6, and FP4 — enabling extremely flexible precision choices.

2.3 CUDA Programming Model

CUDA (Compute Unified Device Architecture) is NVIDIA's programming framework for writing code that runs on GPUs. Every inference runtime — vLLM, TensorRT, PyTorch — ultimately executes CUDA code under the hood.

2.3.1 The Thread Hierarchy

CUDA organizes GPU execution in a strict three-level hierarchy:

- **Thread:** The smallest unit. One thread performs one computation.
- **Block:** A group of up to 1024 threads that share memory and can synchronize.
- **Grid:** A collection of blocks that together run one kernel.

Analogy: School Analogy

Think of a grid as a **school**. Each block is a **classroom**. Each thread is a **student**.

Students in one classroom can pass notes (shared memory). Students in different classrooms cannot talk directly. The principal (GPU scheduler) tells each classroom what lesson to do (*kernel*).

2.3.2 A Simple CUDA Kernel

```
1 // This function runs on the GPU -- one thread per element
2 __global__ void vector_add(float* A, float* B, float* C, int
  N) {
3     // Each thread computes its own index
4     // blockIdx.x = which block am I in?
5     // blockDim.x = how many threads per block?
6     // threadIdx.x = which thread am I within my block?
7     int idx = blockIdx.x * blockDim.x + threadIdx.x;
8
9     // Boundary check: don't go past the array end
10    if (idx < N) {
11        C[idx] = A[idx] + B[idx];    // Each thread adds one
        element
12    }
13 }
14
15 // Called from the CPU (host)
16 void run_vector_add(float* A, float* B, float* C, int N) {
17     int threads_per_block = 256;
18     int num_blocks = (N + threads_per_block - 1) /
        threads_per_block;
```

```

19 // <<< grid_size, block_size >>> launches the kernel
20 vector_add<<<num_blocks, threads_per_block>>>(A, B, C, N);
21 }
22

```

Listing 2.1: A simple CUDA vector addition kernel

2.3.3 Memory Management in CUDA

```

1 // Allocate memory on CPU (host)
2 float* h_A = (float*)malloc(N * sizeof(float));
3
4 // Allocate memory on GPU (device)
5 float* d_A;
6 cudaMalloc(&d_A, N * sizeof(float)); // GPU memory
   allocation
7
8 // Transfer data CPU -> GPU
9 cudaMemcpy(d_A, h_A, N*sizeof(float), cudaMemcpyHostToDevice);
10
11 // Run kernel on GPU
12 my_kernel<<<grid, block>>>(d_A, ...);
13
14 // Transfer results GPU -> CPU
15 cudaMemcpy(h_result, d_result, N*sizeof(float),
   cudaMemcpyDeviceToHost);
16
17 // Free GPU memory
18 cudaFree(d_A);

```

Listing 2.2: CUDA memory allocation and transfer

Memory Transfer is Expensive

Moving data between CPU and GPU via PCIe costs time — typically 16 GB/s on a PCIe 4.0 $\times 16$ connection. This is 100 $\times$ slower than reading from GPU HBM. In production inference, the goal is to keep *everything* on the GPU and

minimize CPU-GPU transfers.

2.3.4 Warps: The Real Execution Unit

While blocks are the logical unit, hardware executes threads in groups of 32 called **warps**. All 32 threads in a warp execute the same instruction simultaneously. If threads in a warp take different code paths (e.g., an if/else), both paths execute sequentially with some threads masked off. This is called **warp divergence** and is a major source of GPU performance loss.

Exercise 2.1: Thread Index Calculation

A CUDA kernel is launched with a grid of 16 blocks, each containing 256 threads.

1. What is the total number of threads?
2. If `blockIdx.x = 5` and `threadIdx.x = 100`, what is the global thread index?
3. If processing an array of 3000 elements, how many threads will be out of bounds and need a guard check?

2.4 FlashAttention: The Kernel That Changed Inference

Before we leave GPU basics, let's preview one of the most important CUDA optimizations ever invented for AI: **FlashAttention**.

The standard attention computation in a Transformer reads the key, query, and value matrices from HBM, computes a large intermediate matrix (the attention scores), writes it back to HBM, then reads it again to apply softmax. This is **extremely memory-bandwidth-bound**.

FlashAttention (Dao et al., 2022) reorganizes this computation so that all intermediate results stay in shared memory. It never writes the large intermediate matrix

to HBM at all. This single change achieves:

- 2 – 4× faster attention on GPU
- 5 – 20× less memory for attention
- Exact same mathematical result as standard attention

FlashAttention is now standard in every production inference engine.

Chapter Summary

GPUs accelerate AI because they perform thousands of operations in parallel. The memory hierarchy (registers → shared memory → HBM) is central to performance: keep data close to compute. CUDA organizes execution as grids of blocks of threads. Tensor Cores handle matrix multiplication at extreme speed. Modern H100 and Blackwell GPUs support FP8/FP4 precision for even higher throughput. FlashAttention shows how smart kernel design can dramatically improve both speed and memory use.

Transformer Inference Mechanics

Learning Objectives

- Understand how Transformer models generate text token by token
- Learn the two phases of inference: prefill and decode
- Understand why decode is memory-bandwidth-bound, not compute-bound
- Grasp the roofline model for compute vs. memory bound analysis

3.1 How Transformers Generate Text

Large Language Models generate text **one token at a time**. A **token** is roughly a word-piece — “inference” might be one token, while “transformers” might be two: “transform” and “ers”.

When you send a prompt to an LLM, here is what happens:

1. Your prompt is tokenized into a list of integer token IDs.
2. The model processes all prompt tokens simultaneously (**prefill phase**).
3. The model generates one output token, then another, then another, until it produces an end-of-sequence token or reaches a length limit (**decode phase**).

Analogy: The Sentence Completion Game

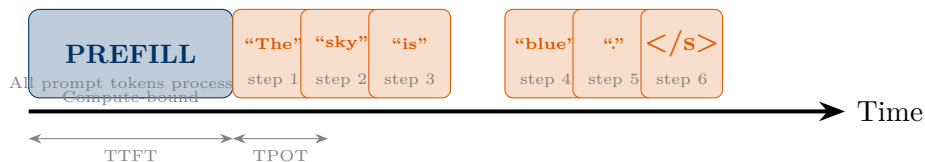
Imagine you are playing a word-completion game. You read the entire sentence so far, then you predict the most likely next word. You write it down. Now you read the whole sentence again, including the word you just wrote, and predict the next word. Repeat.

This is exactly how an LLM works. The entire conversation history is reread every time a new token is generated. This is called *autoregressive* generation.

3.1.1 The Two Phases in Detail

Prefill processes the entire input prompt in a single forward pass. If your prompt has 512 tokens, all 512 are processed at once, using the GPU’s full parallel compute power. Prefill is **compute-bound**: the GPU is busy multiplying matrices.

Decode generates one token per step. At each step, only a single new token is processed (the most recently generated one), while all previous tokens are accessible through the KV cache (Chapter 4). Decode is **memory-bandwidth-bound**: the GPU spends most of its time loading model weights from HBM, not computing.

**3.2 The Roofline Model: Compute vs. Memory Bound**

To understand why decode is slow, we need the **roofline model** — a simple framework for determining whether a computation is limited by arithmetic speed or memory bandwidth.

Every GPU has two fundamental limits:

- **Peak FLOPS:** How many floating-point operations per second it can perform

(H100: ~ 1979 TFLOPS in FP16)

- **Peak Memory Bandwidth:** How fast it can read data from HBM (H100: 3.35 TB/s)

For a given computation, the **arithmetic intensity** is:

$$\text{Arithmetic Intensity} = \frac{\text{FLOP performed}}{\text{bytes read from memory}}$$

If arithmetic intensity is **above** a threshold (called the ridge point), the operation is **compute-bound**: your GPU is fast enough but you're not feeding it enough data.

If arithmetic intensity is **below** the ridge point, the operation is **memory-bandwidth-bound**: your GPU is waiting for data from memory.

For decode with a single request:

- We read the entire model (~ 140 GB for a 70B parameter FP16 model)
- We perform only ~ 280 GFLOPs of computation per token
- Arithmetic intensity ≈ 2 FLOP/byte — far below the H100's ridge point of ~ 590 FLOP/byte

The Decode Efficiency Problem

For a single request, the GPU is roughly **99% idle during decode**. We load 140 GB of model weights to produce maybe 50 bytes of output token. This is deeply inefficient — and it's the core reason batching is so important in inference engineering.

3.3 Inside a Transformer Layer

A single Transformer layer contains two main components:

1. **Multi-Head Self-Attention (MHA):** Each token attends to every other token in the sequence.
2. **Feed-Forward Network (FFN):** A two-layer MLP applied independently to each token.

3.3.1 The Attention Computation

For a sequence of n tokens, the attention computation is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

where Q , K , V are the query, key, and value matrices derived from the input by multiplying with learned weight matrices W_Q , W_K , W_V .

During **decode**, we only compute Q for the new token, but we need all previous K and V values. This is why we need a **KV Cache** (Chapter 4).

3.3.2 Memory Footprint of a Large Model

For a Llama-3 70B model in FP16:

Component	Parameters	Memory (FP16)
Embedding layers	0.5B	1 GB
80 Attention layers	40B	80 GB
80 FFN layers	28B	56 GB
Other (LN, head)	1.5B	3 GB
Total	70B	~140 GB

This model requires **two H100 80GB GPUs** just to load the weights, before any KV cache or batch data.

Chapter Summary

Transformers generate text autoregressively — one token at a time. The prefill phase processes the entire prompt in parallel (compute-bound), while the decode phase generates one token at a time (memory-bandwidth-bound). The roofline model helps understand why: during decode we load all model weights for minimal computation. Batching multiple requests together is the key to making decode efficient. Large models need substantial GPU memory just for their weights.

KV Cache and Memory Management

Learning Objectives

- Understand why the KV cache exists and what it stores
- Learn how PagedAttention eliminates KV cache fragmentation
- Calculate KV cache memory requirements for production systems
- Understand prefix caching and its performance benefits

4.1 Why We Need a KV Cache

Recall the decode phase: we generate one new token at a time, but each token requires attention over *all* previous tokens.

Without caching: at step t , to generate token $t+1$, we would recompute all the key and value vectors for tokens 1 through t . This means the computation grows linearly with sequence length: generating a 1000-token response costs $1 + 2 + 3 + \dots + 1000 = 500,500$ attention computations instead of just 1000.

The **KV Cache** solves this by **storing** the key and value matrices computed for every past token. When generating token $t+1$, we reuse all stored K and V from steps 1 through t and only compute new K and V for the latest token.

Analogy: The Meeting Notes

Imagine you are taking notes in a long meeting. Instead of re-reading the entire transcript every time you want to write the next sentence, you keep a notepad

of the key points discussed so far.

Each time you need to reference the past, you glance at your notepad — not the entire transcript.

The KV cache is that notepad. It stores the important “memory” of past tokens so the GPU doesn’t recompute it every step.

4.2 KV Cache Memory Calculation

The KV cache size for a single sequence is:

$$\text{KV cache size} = 2 \times n_{\text{layers}} \times n_{\text{heads}} \times d_{\text{head}} \times \text{seq_len} \times \text{bytes_per_element}$$

The factor 2 accounts for both keys (K) and values (V).

For Llama-3 70B (FP16, 4096 sequence length):

$$2 \times 80 \times 64 \times 128 \times 4096 \times 2 \approx \mathbf{10.7 \text{ GB}}$$

Memory Shock

A single conversation of 4096 tokens with Llama-3 70B uses **10.7 GB** of KV cache — on top of the 140 GB for weights. Serving 10 simultaneous users requires 107 GB of KV cache alone. Efficient KV cache management is one of the biggest challenges in production inference.

4.3 The Problem: KV Cache Fragmentation

Before PagedAttention, every inference engine allocated KV cache memory using a single large contiguous block per sequence.

Imagine you are running a hotel. Every guest (request) is allocated a room block: a contiguous range of rooms. But you don’t know in advance how long each guest

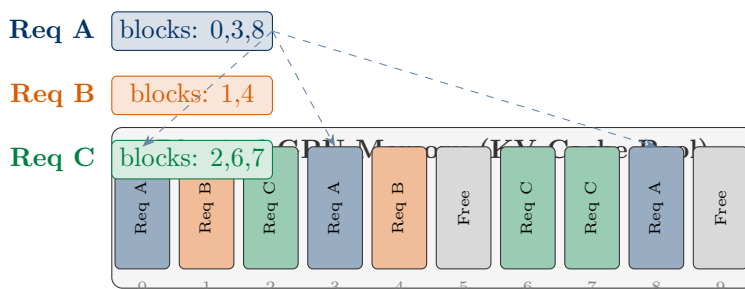
will stay. You must reserve the maximum possible stay (max sequence length) for each guest upfront. The result: many rooms sit empty, but no new guests can be booked because there's no single contiguous block large enough.

In production, this **fragmentation** means that even when 30% of GPU memory is technically free, it cannot be used because it is scattered in small unusable pieces. Studies found that before PagedAttention, 60–80% of GPU memory allocated for KV caches was wasted.

4.4 PagedAttention: Solving Fragmentation

PagedAttention (Kwon et al., SOSP 2023 — the core innovation behind vLLM) borrows an idea from operating system virtual memory: **paging**.

Instead of one contiguous block per request, memory is divided into fixed-size **blocks** (typically 16 or 32 tokens per block). A request uses a **block table** — a list of pointers to the physical blocks it owns. Blocks do not need to be contiguous in memory.



PagedAttention Impact

PagedAttention reduces KV cache waste from 60–80% down to under 5%. vLLM achieves 14–24× higher throughput than naive HuggingFace Transformers and 2.2–3.5× higher than early Text Generation Inference (TGI) engines for LLaMA models on NVIDIA GPUs.

4.5 Prefix Caching

Many queries share common prefixes: the same system prompt, the same document being analyzed, the same few-shot examples.

Prefix caching (also called *prompt caching* or *RadixAttention* in SGLang) stores the KV cache of common prefixes and **reuses** them across multiple requests.

If 1000 users all send queries starting with the same 2000-token system prompt, prefix caching means the KV computation for those 2000 tokens is performed *once*, not 1000 times.

```
1 from vllm import LLM, SamplingParams
2
3 # Initialize vLLM with prefix caching enabled
4 llm = LLM(
5     model="meta-llama/Llama-3-70b-instruct",
6     enable_prefix_caching=True,      # Enable prefix caching
7     gpu_memory_utilization=0.90,    # Use 90% of GPU memory for
        KV pool
8     tensor_parallel_size=2,         # Across 2 GPUs
9 )
10
11 # Many requests sharing the same system prompt will be served
12 # faster because the KV cache for the system prompt is reused
13 system_prompt = "You are a helpful coding assistant. " * 200
        # long prompt
14
15 responses = llm.generate(
16     [system_prompt + user_q for user_q in user_questions],
17     SamplingParams(temperature=0.7, max_tokens=512)
18 )
```

Listing 4.1: Enabling prefix caching in vLLM

4.6 KV Cache Quantization

Since KV cache can consume more memory than the model weights themselves for long sequences, quantizing the KV cache is a powerful optimization.

Recent research shows that INT8 quantization of the KV cache achieves a **4× memory reduction** with reconstruction error below 0.004 and attention score error below 0.1 even for 8K-dimensional heads. FP8 KV quantization is now natively supported in vLLM and TensorRT-LLM.

```
1 from vllm import LLM
2
3 llm = LLM(
4     model="meta-llama/Llama-3-70b-instruct",
5     kv_cache_dtype="fp8",           # Use FP8 for KV cache
6     enable_prefix_caching=True,
7     gpu_memory_utilization=0.92,
8 )
9 # FP8 KV cache reduces memory by ~2x vs FP16
10 # Minimal accuracy degradation in practice
```

Listing 4.2: FP8 KV cache quantization in vLLM

Chapter Summary

The KV cache stores computed keys and values for all past tokens, allowing efficient autoregressive generation without recomputation. KV cache memory grows linearly with sequence length and can easily exceed model weight memory. PagedAttention solves fragmentation by using block-based allocation similar to OS virtual memory, enabling near-zero waste. Prefix caching reuses KV state for shared prompt prefixes. FP8/INT8 quantization of the KV cache reduces memory by 2–4× with minimal accuracy impact.

Quantization: Fitting More Into Less Memory

Learning Objectives

- Understand what quantization is and why it matters for inference
- Learn the difference between INT8, FP8, INT4, FP4 quantization
- Understand post-training quantization (PTQ) vs. quantization-aware training
- Apply AWQ and GPTQ quantization in practice
- Understand the accuracy–speed–memory tradeoffs

5.1 What Is Quantization?

Neural network weights are typically stored in full precision: **FP32** (32 bits per number) or **FP16/BF16** (16 bits per number).

Quantization replaces these high-precision numbers with lower-precision representations — fewer bits per number — that are **good enough** for inference.

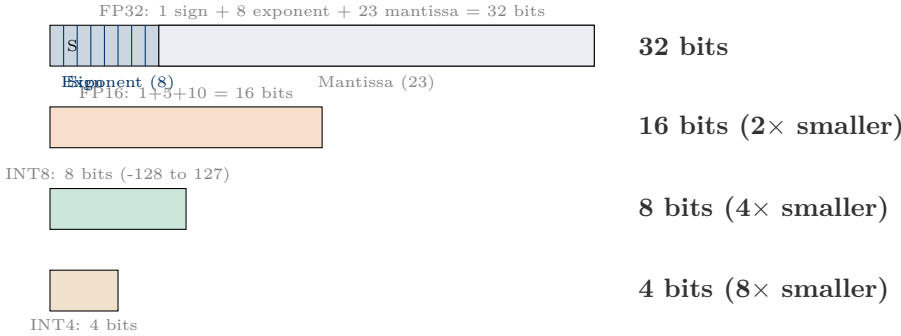
Analogy: The Photo Compression Analogy

Imagine a photograph stored at full resolution: 48 megapixels, 24 bits per pixel. The file is huge — say, 200 MB.

Now you apply JPEG compression. The image now takes 2 MB. When you

look at it on your phone screen, you cannot tell the difference. Quantization is compression for AI model weights. You reduce the number of bits per weight. The model takes less memory, runs faster, and (if done carefully) produces answers that are almost identical to the original.

5.2 Number Formats: A Visual Guide



Format	Bits	Relative Size	Dynamic Range	Use Case
FP32	32	1×	1.2×10^{-38} to 3.4×10^{38}	Training reference
BF16	16	2×	Same range as FP32	Default training/inference
FP16	16	2×	6×10^{-5} to 65504	Common inference
FP8 (E4M3)	8	4×	-448 to 448	H100/Blackwell inference
INT8	8	4×	-128 to 127	Weight/activation quantization

Format	Bits	Relative Size	Dynamic Range	Use Case
INT4/FP4	4	8×	−8 to 7	Aggressive compression

Table 5.1: Number Formats for AI Inference

5.3 Post-Training Quantization Methods

Post-Training Quantization (PTQ) applies quantization to an already trained model, without any additional training.

5.3.1 GPTQ: Round-to-Nearest with Correction

GPTQ (Frantar et al., 2022) is a popular INT4 quantization method that:

1. Quantizes weights layer by layer, in order
2. After quantizing each weight, adjusts remaining weights to compensate for the error introduced
3. Uses a small calibration dataset to guide quantization

```

1 from transformers import AutoModelForCausalLM, AutoTokenizer
2 from optimum.gptq import GPTQQuantizer
3
4 # Load a pre-quantized GPTQ model (4-bit)
5 # Models are available on HuggingFace Hub with "-GPTQ" suffix
6 model = AutoModelForCausalLM.from_pretrained(
7     "TheBloke/Llama-2-70B-GPTQ", # Pre-quantized 4-bit model
8     device_map="auto",          # Auto-distribute across GPUs
9     torch_dtype="auto",
10 )
11 # This 70B model now fits in ~35GB instead of ~140GB

```

Listing 5.1: Loading a GPTQ quantized model

5.3.2 AWQ: Activation-Aware Weight Quantization

AWQ (Lin et al., 2023) is a smarter quantization technique that observes which weights are most **important** for accuracy (by analyzing activations) and protects them from quantization error.

Key insight: *not all weights are equally important*. About 1% of weights are “salient” — they have large activations and dominate the model’s behavior. AWQ keeps these weights at higher precision while aggressively quantizing the rest.

```
1 from vllm import LLM, SamplingParams
2
3 # Load AWQ 4-bit quantized model
4 llm = LLM(
5     model="casperhansen/llama-3-70b-instruct-awq",
6     quantization="awq",           # Tell vLLM this is an AWQ
7     tensor_parallel_size=2,       # Still need 2 GPUs for
8     gpu_memory_utilization=0.92,  4-bit 70B
9 )
10
11 output = llm.generate(
12     ["Explain transformer attention in one sentence."],
13     SamplingParams(temperature=0.7, max_tokens=100)
14 )
15 print(output[0].outputs[0].text)
```

Listing 5.2: Using vLLM with AWQ quantized model

5.3.3 FP8 Quantization on H100/Blackwell

The H100 introduced native FP8 hardware support through its Transformer Engine. FP8 requires minimal post-processing compared to INT quantization because it maintains a floating-point representation, preserving the dynamic range needed by neural network activations.

```

1 from vllm import LLM
2
3 # Enable FP8 quantization (requires H100/Blackwell or newer)
4 llm = LLM(
5     model="meta-llama/Llama-3-70b-instruct",
6     quantization="fp8",           # Use FP8 for both weights
7     gpu_memory_utilization=0.90,  # and activations
8     tensor_parallel_size=1,       # 70B FP8 fits on single
9     H100!
10 )

```

Listing 5.3: FP8 inference with vLLM

5.4 Quantization Accuracy Tradeoffs

Method	Bits	Memory	Speed	Accuracy Loss
BF16 (baseline)	16	140 GB	1.0×	0%
FP8	8	70 GB	1.9×	<0.1%
INT8 (SmoothQuant)	8	70 GB	1.7×	<0.5%
INT4 (AWQ)	4	35 GB	2.5×	1–3%
INT4 (GPTQ)	4	35 GB	2.3×	1–5%
FP4 (Blackwell)	4	35 GB	4.0×	<1%

All numbers are approximate for Llama-3 70B on H100. Accuracy loss measured as perplexity increase on common benchmarks.

Which Quantization to Choose?

- **H100/Blackwell, maximum speed:** FP8 (native hardware support, minimal accuracy loss)
- **Memory constrained, quality matters:** AWQ INT4 (best accuracy at 4-bit)
- **Maximum compression:** GPTQ INT4 with group size 32
- **Edge/CPU deployment:** GGUF with llama.cpp (Q4_K_M is excellent)

Chapter Summary

Quantization reduces model weight precision to save memory and increase speed. Lower precision (FP8, INT4) means smaller models, higher throughput, but some accuracy loss. GPTQ and AWQ are leading 4-bit post-training quantization methods. FP8 is now natively supported on H100 and Blackwell GPUs with negligible accuracy loss. On Blackwell, FP4 delivers 4× the throughput of H100 FP8. Always benchmark accuracy on your specific task before deploying a quantized model.

Speculative Decoding

Learning Objectives

- Understand why speculative decoding speeds up inference
- Learn the draft-verify paradigm
- Understand acceptance rates and when speculation helps
- Apply speculative decoding with vLLM and TensorRT-LLM

6.1 The Bottleneck: Sequential Decode Steps

Recall that decode generates one token at a time. Each step requires a full forward pass through the model. Even if you have a very fast GPU, you cannot parallelize these steps: token $n + 1$ cannot be generated until token n is known.

Analogy: The Draft and Reviewer

Imagine a document that must be approved by an expert (the large model). The expert is slow but always right.

You hire a fast intern (the small model) to write a draft. The intern quickly suggests the next 5 words. Then the expert reviews all 5 words in a single pass — accepting most of them, and correcting any mistakes.

Result: the expert does one review instead of five. If the intern is correct 4 out of 5 times, the overall speed roughly doubles.

This is *speculative decoding*.

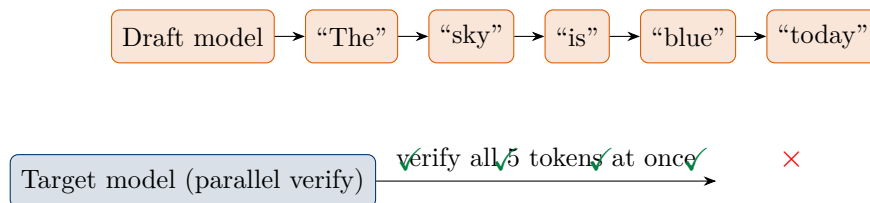
6.2 How Speculative Decoding Works

Speculative decoding uses two models:

- **Draft model:** A small, fast model that quickly generates k candidate tokens.
- **Target model:** The large, accurate model that verifies all k tokens in **one parallel forward pass**.

The key insight is that the target model can verify k tokens in parallel (similar to prefill) rather than generating them sequentially. If the draft is mostly correct, we get multiple output tokens per target model forward pass.

Step 1: Draft 5 tokens



4 tokens accepted, 1 rejected (acceptance rate = 80%)

6.2.1 N-gram and EAGLE Speculation

Beyond a separate small model, modern systems use:

- **N-gram speculation:** Predict next tokens based on n-gram statistics from the current context. Fast and requires no extra model.
- **EAGLE/EAGLE-3:** Uses a shallow draft head that shares the target model's hidden states. Achieves $3\times$ speedup on typical text.

- **Medusa:** Multiple parallel draft heads on the same model backbone.

```

1 from vllm import LLM, SamplingParams
2
3 # Method 1: Use a small draft model
4 llm = LLM(
5     model="meta-llama/Llama-3-70b-instruct",          # Target
6     speculative_model="meta-llama/Llama-3-8b-instruct", # Draft model
7     num_speculative_tokens=5,      # Generate 5 draft tokens per
8     tensor_parallel_size=2,        step
9 )
10
11 # Method 2: N-gram based speculation (no extra model needed)
12 llm_ngram = LLM(
13     model="meta-llama/Llama-3-70b-instruct",
14     speculative_model="[ngram]",      # Built-in n-gram drafter
15     ngram_prompt_lookup_max=4,        # Look back 4 tokens for
16     tensor_parallel_size=2,          patterns
17 )
18
19
20 result = llm.generate(
21     ["Write a Python function to sort a list:"],
22     SamplingParams(temperature=0.0, max_tokens=200) # Greedy
23 )

```

Listing 6.1: Speculative decoding with vLLM

When Speculation Helps Most

Speculative decoding helps most when:

- Output is predictable (code, structured text, repetitive responses)

- Batch size is small (low utilization otherwise)
- Latency matters more than throughput

Speculative decoding *hurts* throughput at high batch sizes because it adds verification overhead. At a batch size of 32+, continuous batching is usually better than speculation.

Chapter Summary

Speculative decoding uses a fast draft model to propose multiple tokens, which the target model verifies in parallel. When the draft is mostly correct, this delivers 2–4× lower latency for single-request serving. EAGLE-3 and n-gram speculation are the preferred methods in 2025. Speculation is most beneficial for low-batch, latency-sensitive use cases, not high-throughput batch serving.

Inference Runtimes: vLLM, TensorRT-LLM, and SGLang

Learning Objectives

- Understand the architecture of the three leading inference runtimes
- Compare vLLM, TensorRT-LLM, and SGLang across key dimensions
- Know which runtime to choose for a given use case
- Deploy a model with vLLM and a REST API

7.1 The Inference Runtime Stack

An **inference runtime** is the software layer that takes a trained model and serves it efficiently to users. Think of it as the kitchen of a restaurant: the model is the recipe, and the runtime is the kitchen with all its equipment, scheduling, and organization.

Every production runtime must handle:

- **Model loading:** Reading weights from disk, placing on GPUs
- **Request queuing:** Accepting HTTP/gRPC requests from users
- **Batching:** Grouping multiple requests together for efficiency
- **Scheduling:** Deciding which requests to process next
- **Memory management:** Allocating/freeing KV cache for each request
- **Output:** Streaming tokens back to users as they are generated

7.2 vLLM: The Production Standard

vLLM (UC Berkeley, 2023, now a PyTorch Foundation project) is the most widely adopted open-source inference engine. Its core innovations:

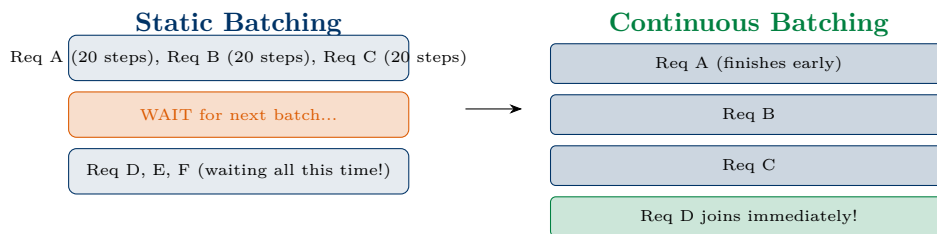
- **PagedAttention:** Eliminates KV cache fragmentation (Chapter 4)
- **Continuous batching:** New requests join as soon as a slot frees up
- **Prefix caching:** Reuses KV state for shared prefixes
- **OpenAI-compatible API:** Drop-in replacement for OpenAI API

Typical performance: 120–160 requests/second, 50–80ms TTFT with 100 concurrent users for Llama-class models on H100.

7.2.1 Continuous Batching

Traditional batching waits for a batch to be full before starting. This is like a bus that only departs when every seat is full — efficient for the bus, but terrible for passengers who arrived early.

Continuous batching (also called iteration-level scheduling) allows new requests to join the batch at every decode step. When request A finishes at step 50, a new request immediately fills that slot.



7.2.2 Running vLLM

```
1  # Install vLLM
2  pip install vllm
3
4  # Start the server (OpenAI-compatible API on port 8000)
5  python -m vllm.entrypoints.openai.api_server \
6      --model meta-llama/Llama-3-70b-instruct \
7      --tensor-parallel-size 2 \           # Use 2 GPUs
8      --gpu-memory-utilization 0.90 \     # Use 90% of GPU
9      memory for KV cache
10     --enable-prefix-caching \           # Enable prefix caching
11     --kv-cache-dtype fp8 \             # FP8 KV cache
12     --max-num-seqs 256 \               # Max concurrent
13     sequences
14     --port 8000
```

Listing 7.1: Starting the vLLM OpenAI-compatible server

```
1  import openai
2
3  # vLLM is OpenAI API compatible
4  client = openai.OpenAI(
5      base_url="http://localhost:8000/v1",
6      api_key="not-needed"                # vLLM doesn't require
7      a key
8  )
9
10 response = client.chat.completions.create(
11     model="meta-llama/Llama-3-70b-instruct",
12     messages=[
13         {"role": "system", "content": "You are a helpful
14         assistant."},
15         {"role": "user", "content": "Explain KV cache in 2
16         sentences."}
17     ],
18     max_tokens=200,
19     stream=True                          # Enable streaming
```

```
17 )  
18  
19 for chunk in response:  
20     if chunk.choices[0].delta.content:  
21         print(chunk.choices[0].delta.content, end="",  
                flush=True)
```

Listing 7.2: Querying the vLLM server

7.3 TensorRT-LLM: Maximum NVIDIA Performance

TensorRT-LLM (NVIDIA) compiles a model into a highly optimized engine specifically for a particular GPU, precision, and sequence configuration.

Key advantages:

- Lowest latency at low concurrency (35–50ms TTFT vs. vLLM’s 50–80ms)
- Deep GPU kernel fusion (multiple operations merged into one kernel)
- FP8/FP4 native support for Blackwell
- Disaggregated serving via NVIDIA Dynamo

Key disadvantages:

- Complex engine compilation (can take hours per model)
- NVIDIA-only (no AMD, Intel)
- Requires re-compilation when model or GPU changes

```
1 # Install TensorRT-LLM  
2 pip install tensorrt_llm  
3  
4 # Step 1: Convert model to TensorRT-LLM checkpoint format  
5 python convert_checkpoint.py \
```

```
6      --model_dir ./llama-3-70b \
7      --output_dir ./trt-checkpoint \
8      --dtype float16
9
10 # Step 2: Build the TRT engine (this compiles for your
11      specific GPU)
12 trtllm-build \
13     --checkpoint_dir ./trt-checkpoint \
14     --output_dir ./trt-engine \
15     --max_input_len 2048 \
16     --max_output_len 512 \
17     --max_batch_size 32 \
18     --tp_size 2                                # Tensor parallel across 2
19                                           GPUs
20
21 # Step 3: Run inference
22 python run.py \
23     --engine_dir ./trt-engine \
24     --max_output_len 100 \
25     --input_text "Explain GPU memory hierarchy:"
```

Listing 7.3: Building and running a TensorRT-LLM engine

7.4 SGLang: Structured Generation Language

SGLang (Zheng et al., 2024) introduces *RadixAttention* — an automatic KV cache reuse system that builds a radix tree of all cached prefixes and shares them optimally across requests.

SGLang excels for:

- Long-document processing (PDF analysis, RAG)
- Conversation-style traffic with long shared history
- Multi-turn agents that reuse the same system context

- Structured generation (JSON output, grammars)

```
1 pip install sglang
2
3 # Launch SGLang server
4 python -m sglang.launch_server \
5     --model-path meta-llama/Llama-3-70b-instruct \
6     --tp 2 \                               # Tensor parallel size
7     --mem-fraction-static 0.82 \
8     --enable-flashinfer                    # Use FlashInfer attention
        backend
```

Listing 7.4: Starting SGLang server

7.5 Runtime Comparison: The Decision Guide

Criterion	vLLM	TRT-LLM	SGLang
Setup time	Minutes	Hours/days	Minutes
TTFT (low concurrency)	50–80ms	35–50ms	50–80ms
Throughput (high concurrency)	Excellent	Excellent	Excellent
Hardware	Any (NVIDIA/A/AMD)	NVIDIA only	NVIDIA/AMD
Prefix caching	Good	Good	Excellent (Radix-Attention)
Model support	Widest	NVIDIA-curated	Growing
API compatibility	OpenAI	Custom/Triton	OpenAI
Best for	General production	Max NVIDIA perf	Long context/RAG

Table 7.1: Inference Runtime Comparison (2025)

Chapter Summary

vLLM is the go-to for most production deployments: easy setup, OpenAI-compatible, excellent throughput. TensorRT-LLM achieves the lowest latency on NVIDIA hardware but requires compilation effort. SGLang leads for long-context workloads with RadixAttention. All three support modern features: FP8, continuous batching, prefix caching. Choose based on your traffic shape, ops maturity, and latency SLAs.

Distributed Inference: Parallelism Strategies

Learning Objectives

- Understand why a single GPU is often not enough
- Learn tensor parallelism, pipeline parallelism, and sequence parallelism
- Understand NVLink and its role in multi-GPU scaling
- Apply tensor parallelism with vLLM

8.1 When One GPU Is Not Enough

A Llama-3 70B model in FP16 requires 140 GB of memory — far more than a single H100's 80 GB. Even smaller models may benefit from multiple GPUs for throughput.

Distributing a model across multiple GPUs requires a **parallelism strategy**. There are three main approaches:

8.2 Tensor Parallelism

Tensor parallelism (TP) splits individual matrices *across* GPUs.

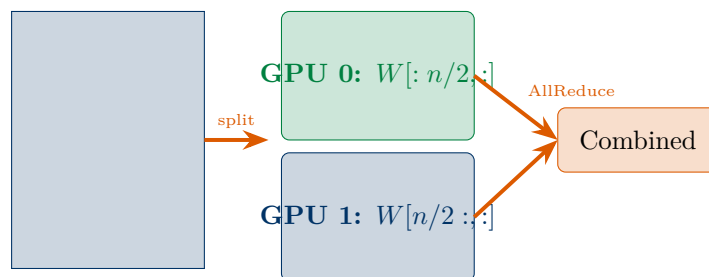
Analogy: The Heavy Book

Imagine a very large book that is too heavy for one person to carry. Instead of one person carrying the whole book, two people each carry half — one carries pages 1–500, the other carries pages 501–1000.

Tensor parallelism works similarly. A large weight matrix (say, 8192×8192) is split across 2 GPUs: each GPU stores a 4096×8192 slice.

Both GPUs compute in parallel, then combine their partial results. The communication (combining results) uses NVLink for very high bandwidth.

With TP-4 (4 GPUs), each GPU holds 1/4 of each weight matrix. Each forward pass requires an **AllReduce** communication operation to combine results across all 4 GPUs.



8.2.1 Launching Tensor-Parallel vLLM

```

1 # 70B model across 2 H100s with TP=2
2 python -m vllm.entrypoints.openai.api_server \
3     --model meta-llama/Llama-3-70b-instruct \
4     --tensor-parallel-size 2           # Split across 2 GPUs
5
6 # 405B model across 8 H100s with TP=8
7 python -m vllm.entrypoints.openai.api_server \
8     --model meta-llama/Llama-3-405b-instruct \
9     --tensor-parallel-size 8           # Split across 8 GPUs

```

Listing 8.1: Running vLLM with tensor parallelism

8.3 Pipeline Parallelism

Pipeline parallelism (PP) assigns different model *layers* to different GPUs. Layers 0–19 run on GPU 0, layers 20–39 run on GPU 1, etc.

Analogy: The Assembly Line

In a car factory, one station installs the engine, another the doors, another the paint. Each station works on a different car simultaneously.

In pipeline parallelism, GPU 0 finishes processing layers 0–19 for batch 1 and passes the intermediate activations to GPU 1. While GPU 1 works on batch 1, GPU 0 starts processing batch 2.

The pipeline keeps all GPUs busy, like a factory assembly line.

Pipeline parallelism communicates via a simpler point-to-point activation transfer (not AllReduce), making it more efficient across slower interconnects (e.g., InfiniBand, PCIe) than tensor parallelism.

8.4 NVLink: The Fast Lane Between GPUs

Communication between GPUs is the bottleneck in distributed inference. NVLink is NVIDIA’s high-speed GPU interconnect, far faster than PCIe:

Interconnect	Bandwidth (each dir.)	Use Case
PCIe 5.0 $\times 16$	64 GB/s	CPU–GPU
NVLink 4 (H100)	900 GB/s	GPU–GPU
NVLink 5 (B200)	1800 GB/s	GPU–GPU
NVSwitch (GB200 NVL72)	57.6 TB/s total	Rack-scale

With NVLink 4 on a DGX H100 system (8 GPUs), the AllReduce bandwidth for tensor parallelism is **28 $\times$** faster than a PCIe-only configuration. This is why TP works well intra-node but poorly across nodes.

The Parallelism Recipe

- **TP within a node** (GPUs on same server, NVLink connected)
- **PP across nodes** (different servers, InfiniBand connected)
- **TP=8, PP=N/8 for very large models** (405B+)

8.5 Disaggregated Prefill and Decode

An emerging production pattern separates the prefill and decode phases onto **different hardware**:

- **Prefill GPUs:** Optimized for high compute (processing long prompts)
- **Decode GPUs:** Optimized for memory bandwidth (fast token generation)

NVIDIA Dynamo (announced GTC 2025) is an orchestration layer that implements this pattern on top of vLLM, TensorRT-LLM, or SGLang. By routing prefill and decode to specialized hardware, it can significantly reduce queue depth and improve both TTFT and throughput simultaneously.

Chapter Summary

Large models require multi-GPU deployment. Tensor parallelism splits weight matrices across GPUs — ideal for intra-node with NVLink. Pipeline parallelism assigns layers to GPUs — better for inter-node. NVLink provides the high-bandwidth interconnect that makes tensor parallelism practical. Disaggregated prefill/decode is an emerging frontier that dedicates different hardware to the two phases of inference.

Observability: Monitoring Inference Systems

Learning Objectives

- Understand the key metrics to monitor in a production inference system
- Learn to use NVIDIA profiling tools: nvtop, nsys, ncu
- Set up Prometheus and Grafana for inference monitoring
- Diagnose common performance bottlenecks from metrics

9.1 The Three Pillars of Observability

A production inference system must be observable:

- **Metrics:** Numerical measurements over time (latency p99, GPU util%)
- **Logs:** Events with timestamps (errors, request traces)
- **Traces:** Request-level timing breakdown (prefill time, queue time)

9.2 Key Metrics to Monitor

Metric	What It Measures	Action if Bad
GPU utilization	% time GPU is active	<60%: add batching; increase load
GPU memory %	VRAM in use	>95%: reduce batch size or add GPUs
TTFT (p50/p95/p99)	Latency to first token	High: check queue depth, pre-fill load
TPOT (p50/p95)	Latency per output token	High: check decode batch size
Throughput (tok/s)	Tokens generated per second	Low: check GPU util, batching
Request queue depth	Requests waiting	>0 for >1s: scale up
KV cache usage %	Cache pool utilization	Near 100%: risk of OOM eviction
Token generation rate	New tokens/s overall	Declining: memory pressure

Table 9.1: Production Inference Monitoring Metrics

9.3 GPU Profiling Tools

9.3.1 nvidia-smi: Real-Time GPU Monitor

```

1 # Install nvidia-smi
2 apt-get install nvidia-smi
3
4 # Run nvidia-smi
5 nvidia-smi
6 # Shows: GPU util%, memory, power, temperature for all GPUs
7 # Press F2 to configure display, q to quit

```

Listing 9.1: Using nvidia-smi for GPU monitoring

9.3.2 Nsight Systems: Timeline Profiling

```
1 # Profile a vLLM inference run
2 nsys profile \
3     --trace cuda,nvtx,osrt \
4     --output inference_profile \
5     python inference_benchmark.py
6
7 # Open in Nsight Systems GUI
8 nsys-ui inference_profile.nsys-rep
9 # Shows: kernel timelines, memory transfers, CPU/GPU overlap
```

Listing 9.2: Profiling with Nsight Systems

9.3.3 Nsight Compute: Kernel-Level Analysis

```
1 # Profile specific CUDA kernels (use on small workloads)
2 ncu --set full \
3     --kernel-name "flash_fwd_kernel" \
4     --export kernel_profile \
5     python run_attention.py
6
7 # Key metrics to look for:
8 # - SM active cycles: is GPU compute actually busy?
9 # - Memory throughput: are we hitting bandwidth limits?
10 # - Warp efficiency: are warps diverging?
```

Listing 9.3: Per-kernel profiling with ncu

9.4 Prometheus and Grafana for Inference

vLLM exposes metrics natively at `/metrics` in Prometheus format:

```
1 python -m vllm.entrypoints.openai.api_server \
2     --model meta-llama/Llama-3-70b-instruct \
```

```
3 --tensor-parallel-size 2 \  
4 --port 8000 \  
5 --metrics-enabled           # Enable Prometheus metrics  
6 # Metrics available at http://localhost:8000/metrics
```

Listing 9.4: Starting vLLM with metrics

```
1 # prometheus.yml  
2 scrape_configs:  
3   - job_name: 'vllm'  
4     static_configs:  
5       - targets: ['localhost:8000']  
6     metrics_path: '/metrics'  
7     scrape_interval: 5s  
8  
9 # Key vLLM metrics:  
10 # vllm:num_requests_running      - active requests  
11 # vllm:gpu_cache_usage_perc     - KV cache % used  
12 # vllm:e2e_request_latency_seconds - end-to-end latency  
13 # vllm:generation_tokens_total  - tokens generated
```

Listing 9.5: Prometheus scrape config for vLLM

Chapter Summary

Production inference requires continuous monitoring. Key metrics are GPU utilization, KV cache usage, TTFT, TPOT, and throughput. NVIDIA tools (nvidia-smi, nsys, ncu) provide GPU-level insight. vLLM's Prometheus metrics enable Grafana dashboards for operational monitoring. High queue depth means scale up. Low GPU utilization means increase batch size or adjust scheduling.

Production Deployment with Kubernetes

Learning Objectives

- Understand how to containerize and deploy inference servers
- Configure GPU resources in Kubernetes
- Implement auto-scaling based on latency metrics
- Set up health checks and rolling updates for zero-downtime deployments

10.1 Containerizing a vLLM Server

```
1  # Dockerfile
2  FROM nvcr.io/nvidia/cuda:12.4.0-devel-ubuntu22.04
3
4  # Install Python and dependencies
5  RUN apt-get update && apt-get install -y python3-pip
6      python3-dev && \
7      pip install vllm==0.7.0 huggingface_hub
8
9  # Pre-download the model (or mount as a volume in production)
10 RUN python3 -c "from huggingface_hub import snapshot_download;
11 \
12     snapshot_download('meta-llama/Llama-3-8b-instruct')"
13
14 # Expose the API port
15 EXPOSE 8000
16
17 # Start the vLLM server
```

```
16 CMD ["python3", "-m", "vllm.entrypoints.openai.api_server", \  
17     "--model", "meta-llama/Llama-3-8b-instruct", \  
18     "--port", "8000", \  
19     "--gpu-memory-utilization", "0.90"]
```

Listing 10.1: Dockerfile for a vLLM inference server

10.2 Kubernetes Deployment

```
1 # vllm-deployment.yaml  
2 apiVersion: apps/v1  
3 kind: Deployment  
4 metadata:  
5   name: vllm-llama3-8b  
6   labels:  
7     app: vllm-inference  
8 spec:  
9   replicas: 2                                # Start with 2 replicas  
10  selector:  
11    matchLabels:  
12      app: vllm-inference  
13  template:  
14    metadata:  
15      labels:  
16        app: vllm-inference  
17    spec:  
18      containers:  
19        - name: vllm  
20          image: my-registry/vllm-llama3:latest  
21          ports:  
22            - containerPort: 8000  
23          resources:  
24            limits:  
25              nvidia.com/gpu: "1"            # 1 GPU per pod  
26              memory: "40Gi"  
27              cpu: "8"
```

```
28         requests:
29             nvidia.com/gpu: "1"
30             memory: "32Gi"
31             cpu: "4"
32     env:
33     - name: CUDA_VISIBLE_DEVICES
34       value: "0"
35     livenessProbe:
36         httpGet:
37             path: /health
38             port: 8000
39             initialDelaySeconds: 60
40             periodSeconds: 30
41     readinessProbe:
42         httpGet:
43             path: /health
44             port: 8000
45             initialDelaySeconds: 90
46             periodSeconds: 10
47     ---
48     apiVersion: v1
49     kind: Service
50     metadata:
51         name: vllm-service
52     spec:
53         selector:
54             app: vllm-inference
55         ports:
56         - port: 80
57           targetPort: 8000
58         type: LoadBalancer
```

Listing 10.2: Kubernetes Deployment for vLLM

10.3 Horizontal Pod Autoscaler

```
1 # hpa-vllm.yaml
2 apiVersion: autoscaling/v2
3 kind: HorizontalPodAutoscaler
4 metadata:
5   name: vllm-hpa
6 spec:
7   scaleTargetRef:
8     apiVersion: apps/v1
9     kind: Deployment
10    name: vllm-llama3-8b
11  minReplicas: 2
12  maxReplicas: 10
13  metrics:
14  - type: Pods
15    pods:
16      metric:
17        name: vllm_queue_depth      # Scale when queue is growing
18      target:
19        type: AverageValue
20        averageValue: "5"          # Scale up when avg queue >
                                   5 requests
```

Listing 10.3: HPA based on custom latency metric

```
1 # Apply deployment
2 kubectl apply -f vllm-deployment.yaml
3
4 # Check pod status
5 kubectl get pods -l app=vllm-inference
6
7 # Check GPU allocation
8 kubectl describe pod vllm-llama3-8b-<pod-id>
9
10 # View logs
11 kubectl logs -f deployment/vllm-llama3-8b
12
13 # Check service
14 kubectl get service vllm-service
```

Listing 10.4: Deploying to Kubernetes

Chapter Summary

Production inference runs on Kubernetes with GPU-aware pods. Containerize with NVIDIA CUDA base images and expose a health endpoint. Specify `nvidia.com/gpu` resource limits. Use HPA with custom metrics (queue depth, latency) for autoscaling. Set readiness probes with long initial delays — model loading takes time. Use rolling updates for zero-downtime deployments.

PART II

Advanced Topics

Long Context, Multimodal, and Agentic Inference

Learning Objectives

- Understand the unique challenges of long-context inference
- Learn how multimodal models handle images, audio, and video
- Understand agentic inference patterns and their infrastructure needs

11.1 Long-Context Inference

Models with 128K–1M token context windows introduce extreme memory and compute challenges. The KV cache for a single 128K-token conversation with Llama-3 70B reaches:

$$2 \times 80 \times 64 \times 128 \times 131072 \times 2 \approx \mathbf{344 \text{ GB}}$$

This dwarfs even the GPU’s HBM capacity. Techniques for long-context efficiency:

- **Sparse attention:** Only attend to a subset of past tokens (e.g., StreamingLLM’s “attention sinks”)
- **KV cache quantization to INT2/INT4:** Sub-4-bit KV compression
- **KV cache offloading:** Spill to CPU memory or NVMe SSD
- **Chunked prefill:** Process long prompts in chunks to control peak memory

```
1 from vllm import LLM, SamplingParams
2
3 llm = LLM(
4     model="meta-llama/Llama-3-70b-instruct",
5     max_model_len=131072,           # 128K context window
6     enable_chunked_prefill=True,    # Process long prompts in
7                                     # chunks
8     max_num_batched_tokens=2048,    # Process 2K tokens per
9                                     # prefill chunk
10    tensor_parallel_size=4,          # Need 4 GPUs for 128K KV
11                                     # cache
12 )
```

Listing 11.1: Chunked prefill for long contexts in vLLM

11.2 Multimodal Inference

Models like LLaVA, Qwen-VL, and InternVL accept image inputs alongside text. The inference pipeline has an extra step: image encoding.

1. **Vision encoder:** A ViT (Vision Transformer) encodes the image into feature tokens — typically 256–1024 tokens per image.
2. **Projection:** A linear layer maps image features to the LLM’s token space.
3. **LLM decode:** The LLM generates text given the image tokens and text prompt.

The bottleneck is that each image adds many tokens to the effective sequence length, inflating KV cache requirements significantly.

11.3 Agentic Inference

AI agents run multiple LLM calls in a loop: plan, act, observe, repeat. The infrastructure implications:

- **Long conversations:** Context accumulates over many steps. Efficient prefix caching is essential.
- **Tool calling:** The model generates structured JSON for tool calls. Low latency per step matters for responsiveness.
- **Parallel function calls:** Some frameworks call multiple tools simultaneously. The serving infrastructure must handle bursty traffic.
- **Request tracing:** Agents make many sub-requests. Tracing is essential for debugging and cost attribution.

Chapter Summary

Long-context inference strains KV cache memory and requires chunked prefill, sparse attention, or quantized KV. Multimodal models add a vision encoder step that contributes many extra tokens. Agentic workloads create bursty, highly correlated requests that benefit from aggressive prefix caching and low per-call latency.

APPENDIX A

CUDA Cheat Sheet

Essential CUDA API

```
1  /*      Memory
   *
2  cudaMalloc(&ptr, bytes);          // Allocate GPU memory
3  cudaFree(ptr);                   // Free GPU memory
4  cudaMemcpy(dst, src, bytes, kind); // Transfer: kind =
5  // cudaMemcpyHostToDevice | cudaMemcpyDeviceToHost |
   DeviceToDevice
6  cudaMemset(ptr, value, bytes);    // Fill GPU memory
7
8  /*      Async memory (for overlap with compute)
   *
9  cudaMallocAsync(&ptr, bytes, stream);
10 cudaFreeAsync(ptr, stream);
11 cudaMemcpyAsync(dst, src, bytes, kind, stream);
12
13 /*      Kernel launch
   *
14 kernel<<<grid, block, shared_mem, stream>>>(args...);
15 // grid = dim3(gridX, gridY, gridZ)
16 // block = dim3(blockX, blockY, blockZ)
17 // max threads per block: 1024
18
19 /*      Thread indexing inside kernel
   *

```

```
20 int idx = blockIdx.x * blockDim.x + threadIdx.x;
21 int idy = blockIdx.y * blockDim.y + threadIdx.y;
22
23 /*      Synchronization
24
25 */
26 cudaDeviceSynchronize();           // Wait for ALL GPU work
27 cudaStreamSynchronize(stream);     // Wait for one stream
28 __syncthreads();                   // Sync threads within
29     block (in kernel)
30
31 /*      Error checking (always do this!)
32
33 */
34 #define CUDA_CHECK(call) { \
35     cudaError_t err = call; \
36     if (err != cudaSuccess) { \
37         fprintf(stderr, "CUDA error: %s at %s:%d\n", \
38             cudaGetErrorString(err), __FILE__, __LINE__); \
39         exit(1); \
40     } \
41 }
42 CUDA_CHECK(cudaMalloc(&ptr, bytes));
43
44 /*      Streams
45
46 */
47 cudaStream_t stream;
48 cudaStreamCreate(&stream);
49 cudaStreamDestroy(stream);
50
51 /*      Device query
52
53 */
54 cudaDeviceProp prop;
55 cudaGetDeviceProperties(&prop, device_id);
56 // prop.name, prop.totalGlobalMem, prop.multiProcessorCount
57 // prop.maxThreadsPerBlock, prop.sharedMemPerBlock
```

Listing A.1: CUDA Cheat Sheet

Performance Rules of Thumb

- Use at least 128 threads per block; 256 is usually optimal
- Ensure memory accesses are coalesced (stride-1 access by consecutive threads)
- Avoid warp divergence in performance-critical paths
- Use shared memory for data reused >2 times within a block
- Overlap compute and memory transfers using CUDA streams
- Profile with nsys before optimizing — never guess the bottleneck

APPENDIX B

Linux Commands for AI Engineers

```
1  #           GPU monitoring
2  nvidia-smi                               # GPU status (util, mem,
    temp)
3  nvidia-smi dmon -s u                     # Live utilization every 1s
4  nvidia-smi --query-gpu=name,memory.total,memory.free \
5      --format=csv                         # GPU memory info
6  watch -n 1 nvidia-smi                   # Auto-refresh every 1s
7  nvidia-smi                               # Interactive GPU monitor
8
9  #           Process management
10 htop                                     # Interactive process
    viewer
11 ps aux | grep python                    # Find Python processes
12 kill -9 <pid>                           # Force kill process
13 nohup python server.py &               # Run in background
14
15 #           Disk and memory
16 df -h                                   # Disk space
17 du -sh /models/*                       # Model directory sizes
18 free -h                                # RAM usage
19 cat /proc/meminfo | grep HugePages    # Huge pages (for pinned
    memory)
20
21 #           Network
22 ss -tlnp | grep 8000                   # Check if port 8000 is open
23 curl http://localhost:8000/health      # Test health endpoint
```

```
24 curl http://localhost:8000/v1/models # List loaded models
25
26 # Benchmarking
27
28 # Time a command
29 time python inference_benchmark.py
30
31 # Test vLLM throughput
32 python -m vllm.entrypoints.benchmark_throughput \
33     --model meta-llama/Llama-3-8b-instruct \
34     --input-len 512 --output-len 128 \
35     --num-prompts 1000
36
37 # Environment
38
39 echo $CUDA_VISIBLE_DEVICES # Which GPUs are visible?
40 export CUDA_VISIBLE_DEVICES=0,1 # Restrict to GPU 0 and 1
41 env | grep CUDA # All CUDA env variables
42
43 # Docker/Container
44
45 docker run --gpus all \ # Use all GPUs in container
46     -p 8000:8000 \
47     my-vllm-image
48
49 # Tmux for long-running jobs
50
51 tmux new -s inference # New session named
52     "inference"
53 tmux attach -t inference # Reattach to session
54 # Ctrl+b, d to detach (session keeps running)
```

Listing B.1: Essential Linux commands for AI engineers

Benchmarking Toolkit

```

1  """
2  AI Inference Benchmarking Toolkit
3  Measures: TTFT, TPOT, throughput, memory usage
4  """
5  import time
6  import asyncio
7  import aiohttp
8  import statistics
9  import json
10 from dataclasses import dataclass, field
11 from typing import List
12
13 @dataclass
14 class BenchmarkResult:
15     ttft_ms: List[float] = field(default_factory=list)
16     tpot_ms: List[float] = field(default_factory=list)
17     total_tokens: List[int] = field(default_factory=list)
18     errors: int = 0
19
20     def summary(self):
21         return {
22             "TTFT_p50_ms": statistics.median(self.ttft_ms),
23             "TTFT_p95_ms": statistics.quantiles(self.ttft_ms,
24             n=20)[18],
25             "TTFT_p99_ms": statistics.quantiles(self.ttft_ms,
26             n=100)[98],
27             "TPOT_p50_ms": statistics.median(self.tpot_ms),
28             "TPOT_p95_ms": statistics.quantiles(self.tpot_ms,
29             n=20)[18],
30             "throughput_tok_s": sum(self.total_tokens) /

```

```
28         (max(self.ttft_ms) / 1000 +
29          sum(self.tpot_ms)/1000),
30         "errors": self.errors,
31     }
32
33 async def benchmark_request(
34     session: aiohttp.ClientSession,
35     url: str,
36     prompt: str,
37     max_tokens: int,
38     result: BenchmarkResult
39 ):
40     payload = {
41         "model": "your-model",
42         "messages": [{"role": "user", "content": prompt}],
43         "max_tokens": max_tokens,
44         "stream": True
45     }
46     start = time.perf_counter()
47     first_token_time = None
48     token_count = 0
49
50     try:
51         async with session.post(f"{url}/v1/chat/completions",
52                                json=payload) as resp:
53             async for line in resp.content:
54                 line = line.decode().strip()
55                 if line.startswith("data: ") and line !=
56                     "data: [DONE]":
57                     chunk = json.loads(line[6:])
58                     delta = chunk["choices"][0]["delta"]
59                     if delta.get("content"):
60                         if first_token_time is None:
61                             first_token_time =
62                                 time.perf_counter()
63                             result.ttft_ms.append(
64                                 (first_token_time - start) *
65                                 1000)
```

```

63         token_count += 1
64
65     if token_count > 1:
66         elapsed = time.perf_counter() - first_token_time
67         result.tpot_ms.append(elapsed * 1000 /
68                               (token_count - 1))
69         result.total_tokens.append(token_count)
70     except Exception as e:
71         result.errors += 1
72         print(f"Error: {e}")
73
74 async def run_benchmark(
75     url: str = "http://localhost:8000",
76     concurrent_users: int = 10,
77     num_requests: int = 100,
78     prompt: str = "Write a 200-word essay about GPU
79                  architecture.",
80     max_tokens: int = 300
81 ):
82     result = BenchmarkResult()
83     connector = aiohttp.TCPConnector(limit=concurrent_users *
84                                     2)
85
86     async with aiohttp.ClientSession(connector=connector) as
87         session:
88         tasks = [
89             benchmark_request(session, url, prompt,
90                               max_tokens, result)
91             for _ in range(num_requests)
92         ]
93         # Run with limited concurrency
94         semaphore = asyncio.Semaphore(concurrent_users)
95         async def bounded_task(t):
96             async with semaphore:
97                 await t
98         await asyncio.gather(*[bounded_task(t) for t in tasks])
99
100     print("\n=== Benchmark Results ===")

```

```
96     for k, v in result.summary().items():
97         print(f"    {k:25s}: {v:.2f}" if isinstance(v, float)
98               else
99               f"    {k:25s}: {v}")
100 if __name__ == "__main__":
101     asyncio.run(run_benchmark(
102         url="http://localhost:8000",
103         concurrent_users=20,
104         num_requests=200,
105     ))
```

Listing C.1: Comprehensive inference benchmarking script

GPU Profiling Tools

Profiling Workflow

```

1  #           Step 1: Quick health check

2  nvidia-smi
3  # Look for: GPU util%, memory used, temperature, power draw
4
5  #           Step 2: Find the bottleneck with nsys

6  nsys profile \
7      --trace=cuda,nvtx,osrt,cudnn,cublas \
8      --sample=cpu \
9      --output=my_profile \
10     python my_inference.py
11
12 # Open GUI
13 nsys-ui my_profile.nsys-rep
14 # KEY QUESTIONS:
15 # - Are there gaps between kernels? (CPU-bound, batching issue)
16 # - Is cudaMemcpy dominating? (unnecessary H2D/D2H transfers)
17 # - Are kernels short? (need to increase batch size)
18
19 #           Step 3: Deep kernel analysis with ncu

20 # WARNING: ncu is slow. Use on small workloads only.
21 ncu --set full \
22     --kernel-name "cutlass" \      # Target specific kernel
23     --launch-skip 10 \            # Skip first 10 launches
                                   (warmup)

```

```
24      --launch-count 3 \           # Profile 3 launches
25      -o kernel_analysis \
26      python run_matmul.py
27
28 ncu-ui kernel_analysis.ncu-rep
29 # KEY METRICS:
30 # - Memory Throughput: are we hitting bandwidth ceiling?
31 # - SM Active Cycles %: is compute busy?
32 # - L2 Hit Rate: are we reusing data?
33 # - Warp Stall Reasons: what's blocking warps?
34
35 #           Step 4: MFU (Model FLOP Utilization)
36
37 # MFU = actual FLOP/s / peak FLOP/s
38 # H100 peak FP16: 989 TFLOP/s
39 # Good MFU for inference: >30% (memory-bound, expected to be
40   low)
41 # Good MFU for large batches: >50%
42
43 #           Step 5: Memory analysis
44
45 # Check for memory fragmentation
46 curl http://localhost:8000/metrics | grep cache
47 # vllm:gpu_cache_usage_perc -> should be <90%
48 # vllm:gpu_cache_hit_rate   -> higher is better (prefix
49   caching)
```

Listing D.1: Full GPU profiling workflow

Interpreting Common Profiling Issues

Symptom	Likely Cause	Fix
GPU util < 30%	Batch too small	Increase max_num_seqs

Symptom	Likely Cause	Fix
Memory > 95%	KV cache overflow	Reduce seq length or add GPUs
TTFT spikes	Large prefill batches	Enable chunked prefill
TPOT high	Memory bandwidth bound	Normal for small batches; use quantization
Frequent OOM	Memory fragmentation	Use PagedAttention (vLLM)
Slow decode after long prefill	KV cache eviction	Increase GPU memory fraction

Table D.1: Common Profiling Issues and Fixes

Research Papers Every Inference Engineer Should Read

Foundational Papers

Attention Is All You Need (Vaswani et al., 2017) The original Transformer paper. Understand this before anything else. <https://arxiv.org/abs/1706.03762>

FlashAttention (Dao et al., 2022) IO-aware exact attention that became standard in all production systems. <https://arxiv.org/abs/2205.14135>

FlashAttention-2 (Dao, 2023) Further optimizations: $2\times$ speedup, better parallelism. <https://arxiv.org/abs/2307.08691>

Efficient Memory Management for LLM Serving with PagedAttention (Kwon et al., 2023) Foundation of vLLM. Must-read for KV cache management. <https://arxiv.org/abs/2309.06180>

Quantization

LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale (Dettmers et al., 2022) First practical INT8 quantization for LLMs. <https://arxiv.org/abs/2208.07339>

GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers (Franssen et al., 2022) State-of-the-art INT4 PTQ. <https://arxiv.org/abs/2210.17323>

AWQ: Activation-aware Weight Quantization (Lin et al., 2023) Better accuracy than GPTQ at same bit width. <https://arxiv.org/abs/2306.00978>

SmoothQuant (Xiao et al., 2022) Smooth activation outliers for INT8 quantization. <https://arxiv.org/abs/2211.10438>

Speculative Decoding

Accelerating Large Language Model Decoding with Speculative Sampling (Chen et al., 2023) Original speculative decoding paper. <https://arxiv.org/abs/2302.01318>

EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty (Li et al., 2023) 3× speedup with draft head sharing target model features. <https://arxiv.org/abs/2401.15077>

Parallelism and Distributed Inference

Megatron-LM (Shoeybi et al., 2019) Efficient tensor and pipeline parallelism for large models. <https://arxiv.org/abs/1909.08053>

Alpa: Automating Inter- and Intra-Operator Parallelism (Zheng et al., 2022) Automatic parallelism for large models. <https://arxiv.org/abs/2201.12023>

Systems and Scheduling

Orca: A Distributed Serving System for Transformer-Based Generative Models (Yu et al., 2022) First continuous batching paper for LLM serving. <https://www.usenix.org/conference/osdi22/presentation/yu>

SGLang: Efficient Execution of Structured Language Model Programs (Zheng et al., 2023) RadixAttention for prefix sharing. <https://arxiv.org/abs/2312.07104>

DejaVu: Contextual Sparsity for Efficient LLMs at Inference Time (Liu et al., 2023)
Dynamic sparsity for faster inference. <https://arxiv.org/abs/2310.17157>

2025 Frontier Papers

TokenWeave: Reducing Tensor-Parallel Communication Overhead (arXiv 2505.11325)
1.2× latency improvement in tensor-parallel low-latency inference.

QuantSpec: Self-Speculative Decoding with Hierarchical Quantized KV Cache (arXiv 2505.11325)
1.78× throughput via combined speculative decoding and KV quantization.

GPU-Accelerated INT8 Quantization for KV Cache Compression (arXiv 2601.04719)
4× memory reduction with CUDA kernels achieving 1694× CPU speedup.

Index

agentic inference, 77
autoregressive decoding, 29
AWQ, 44

B200, 23
Blackwell, 23
block, 24

continuous batching, 52
CPU, 19
CUDA, 23

decode, 30
disaggregated serving, 62

FlashAttention, 26
FP8, 45

GPTQ, 43
GPU, 19
GPU profiling, 64
Grafana, 65
grid, 24

H100, 23
HBM, 21

inference, 13
inference runtime, 51

KV cache, 35
KV cache quantization, 39

latency, 15
long context, 75

memory, 15
memory hierarchy, 21
multimodal inference, 76

Nsight Compute, 65
Nsight Systems, 65
NVLink, 61

PagedAttention, 37
pipeline parallelism, 61
prefill, 30
prefix caching, 38
Prometheus, 65
PTQ, post-training quantization, 43

quantization, 41

rooﬂine model, 30

SGLang, 55
SM, Streaming Multiprocessor, 20
speculative decoding, 48

Tensor Cores, 22

tensor parallelism, 59

TensorRT-LLM, 54

thread, 24

throughput, 15

token, 29

TPOT, 15

training, 13

Transformer layer, 31

TTFT, 15

vLLM, 52

warp, 26

warp divergence, 26